Dynamic Graph Algorithms

CSC-531

Incremental DFS in Undirected Graphs

Dr. Shahbaz Khan
Department of Computer Science and Engineering,
Indian Institute of Technology Roorkee
shahbaz.khan@cs.iitr.ac.in

Question

Think about **incremental maintenance of DFS** trees.

Directed acyclic graph $\boxed{O(mn)}$ Inc. Reach

Undirected (Ensure ***monotonic fall)*** Dec. BFS

Directed graph $O(mn) \rightarrow O(n^2)$

(Think of possible difficulty)

Paper Reference

Franciosa et al. 94

Incremental DFS in Directed Graphs
Ponder over extending the algorithm for DAG's to directed graphs.

What needs to be modified?
Where does the analysis vary?

Depth First Search Trees

Tree edges and Non-Tree edges

2. For a DFS tree of an undirected graph, the non-tree edges can be (0 point)

31% of respondents (10 of 32) answered this question correctly.

More Details

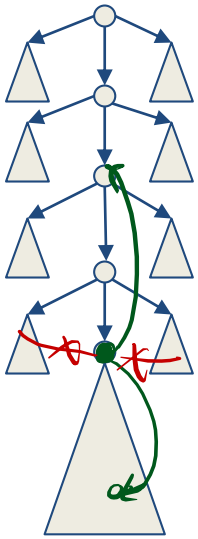
- From a vertex to all its ancestors 25 ✓
- From a vertex to all its descenda... 11 ✓
- From a vertex to any vertex on i... 10
- From a vertex to any vertex on i... 7

From a vertex to all its ancestors ✓

From a vertex to all its descendants ✓

From a vertex to any vertex on its left in the tree

From a vertex to any vertex on its right in the tree



Non-Tree edges?

Undirected Graph: Forward Edges, Back Edges

Incremental Depth First Search Trees (Undirected)

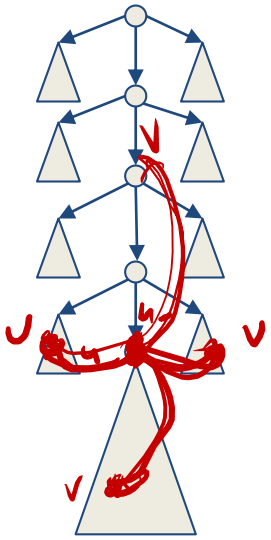
Given an undirected graph G, maintain its DFS under insertion of edges.

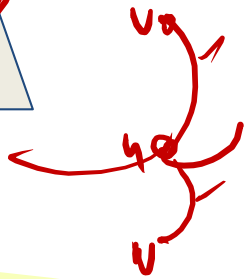
Non-Tree edges? Back Edges

Insertion of Edge (u,v)?

Back Edge (Ancestor-Descendant)? ✗

Cross Edge (Remaining)? ✓





Detect: DFS Start $(u) \geq (v)$ ✗

$\leq$

DFS Finish $u \lesseqgtr v$ Back

Back $S(v) < S(u) < F(u) < F(v)$

$S(u) < S(v) < F(v) < F(u)$

Cross $S(v) < F(v) < S(u) < F(u)$ || $S(u) < F(u) < S(v) < F(v)$

Incremental Depth First Search Trees (Undirected)

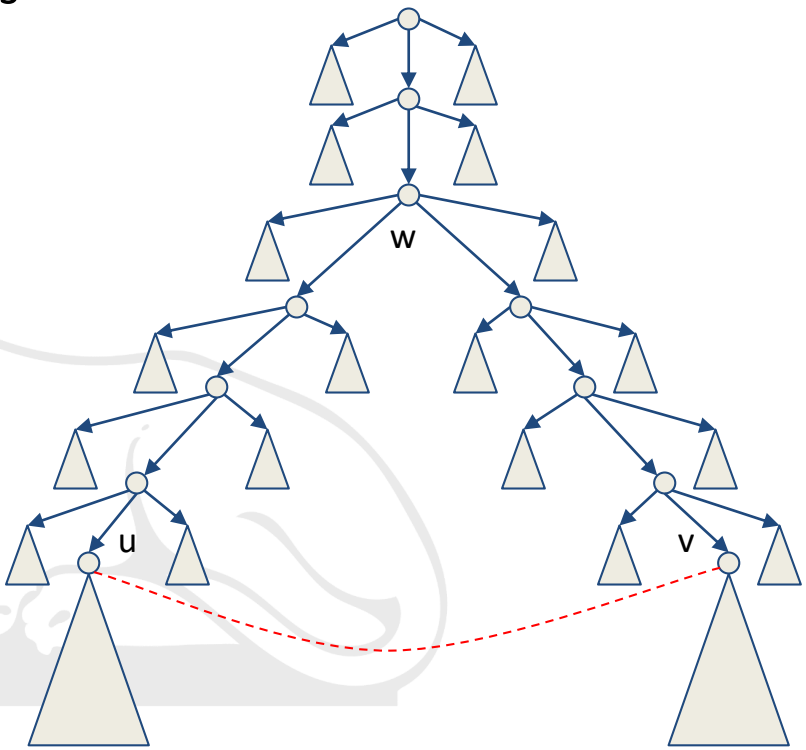
Given an undirected graph G, maintain its DFS under insertion of edges.

Non-Tree edges? Back Edges

Insertion of Edge (u,v)?
Back Edge (Ancestor-Descendant)?
Cross Edge (Remaining)?

Detection?
Using DFS Visit Time
Using DFS Finish Time

DFS Visit[u] < DFS Visit[v]
and
DFS Finish[u] < DFS Finish[v]



ALTERNATE VIEW?

Lowest Common Ancestor

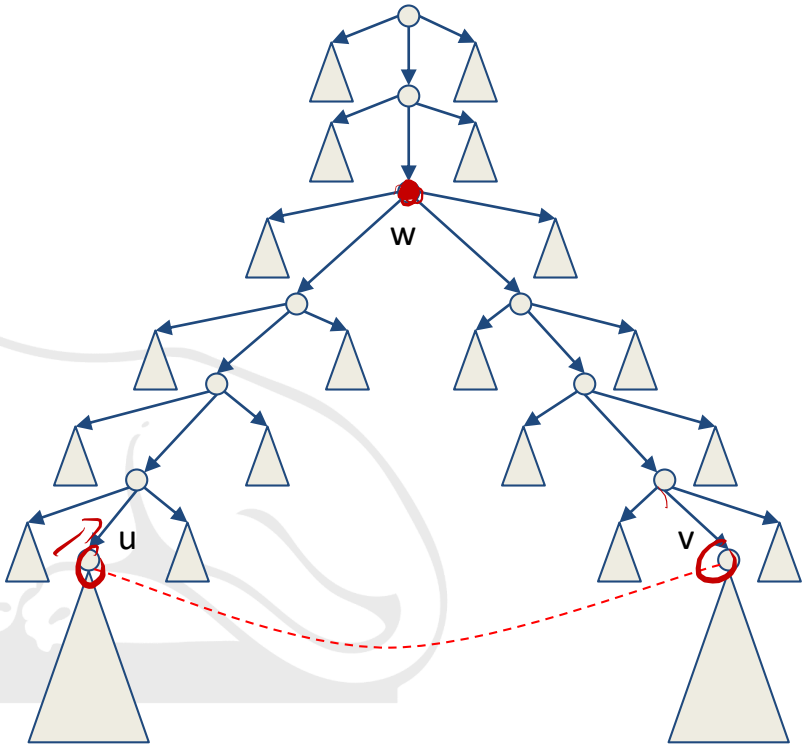
Definition:

LCA(u,v): Lowest vertex which is an ancestor of both u and v

State of the Art?

Query?

Space?



Lowest Common Ancestor

Definition:

LCA(u,v): Lowest vertex which is an ancestor of both u and v

State of the Art:

Static:

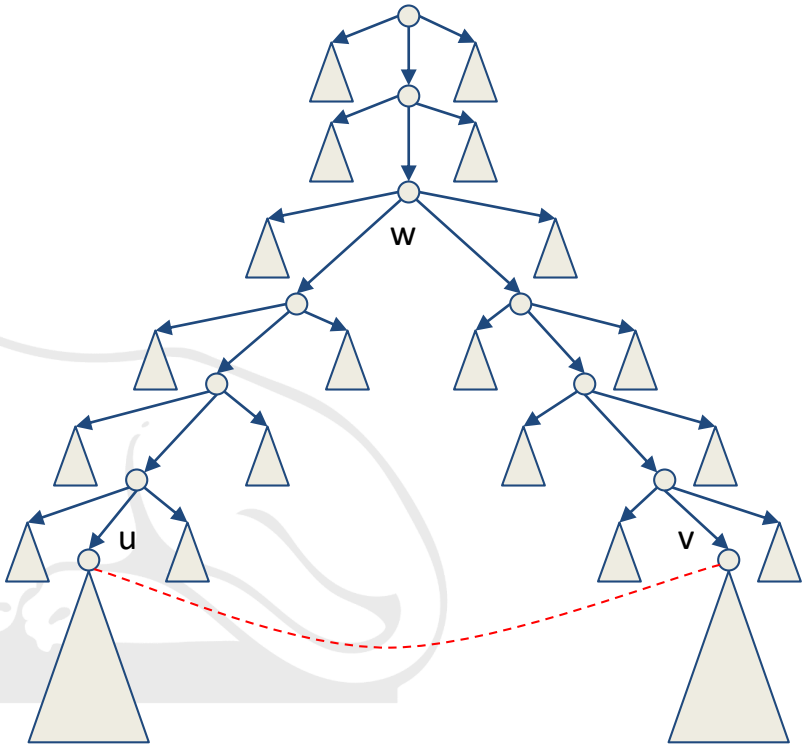
Dov Harel and Robert Tarjan (1984)

(Heavy Path Decomposition)

Preprocess O(n)

Query O(1)

Dynamic?



Lowest Common Ancestor

Definition:

LCA(u,v): Lowest vertex which is an ancestor of both u and v

State of the Art:

Static:

Dov Harel and Robert Tarjan (1984)

(Heavy Path Decomposition)

Preprocess O(n)

Query O(1)

Dynamic:

Richard Cole and Ramesh Hariharan (2005)

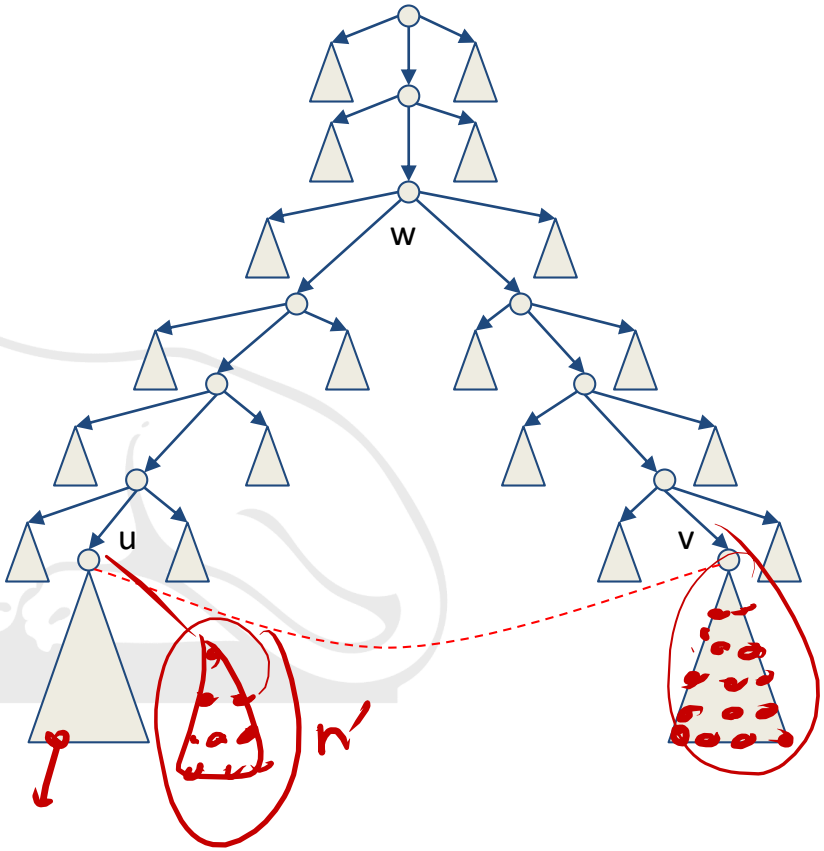
Preprocess O(n)

Query O(1)

Insert Leaf O(1)

Delete Leaf O(1)

O(n')



Incremental Depth First Search Trees (Undirected)

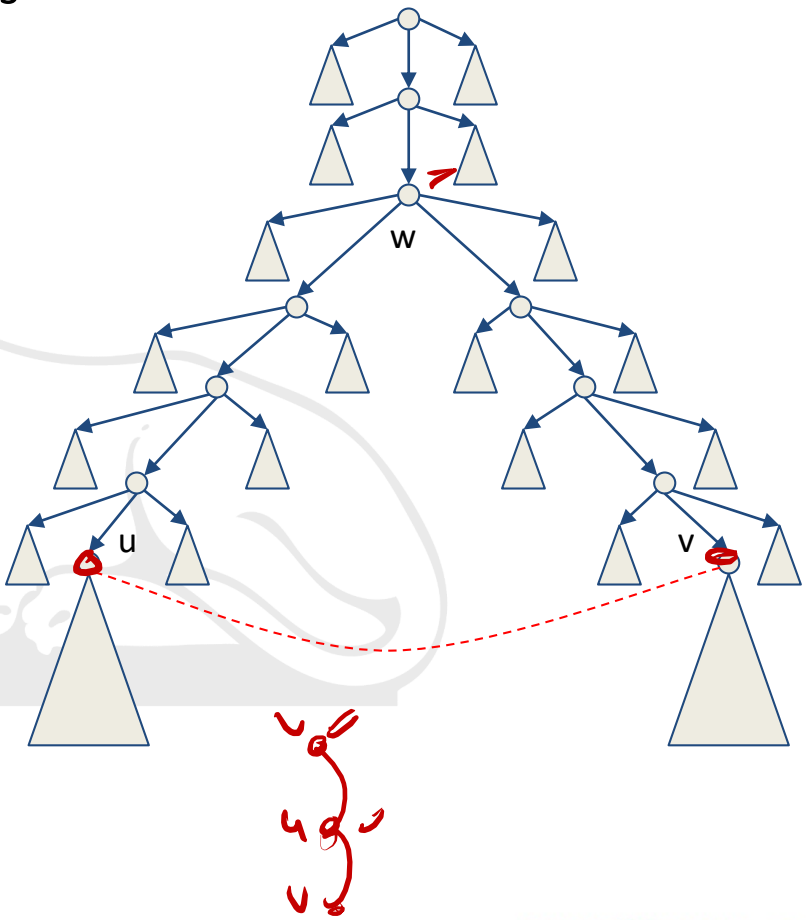
Given an undirected graph G, maintain its DFS under insertion of edges.

Non-Tree edges? Back Edges

Insertion of Edge (u,v)?
Back Edge (Ancestor-Descendant)?
Cross Edge (Remaining)?

Detection?
Using DFS Visit Time
Using DFS Finish Time
DFS Visit[u] < DFS Visit[v]
and
DFS Finish[u] < DFS Finish[v]

Using LCA Query?



Incremental Depth First Search Trees (Undirected)

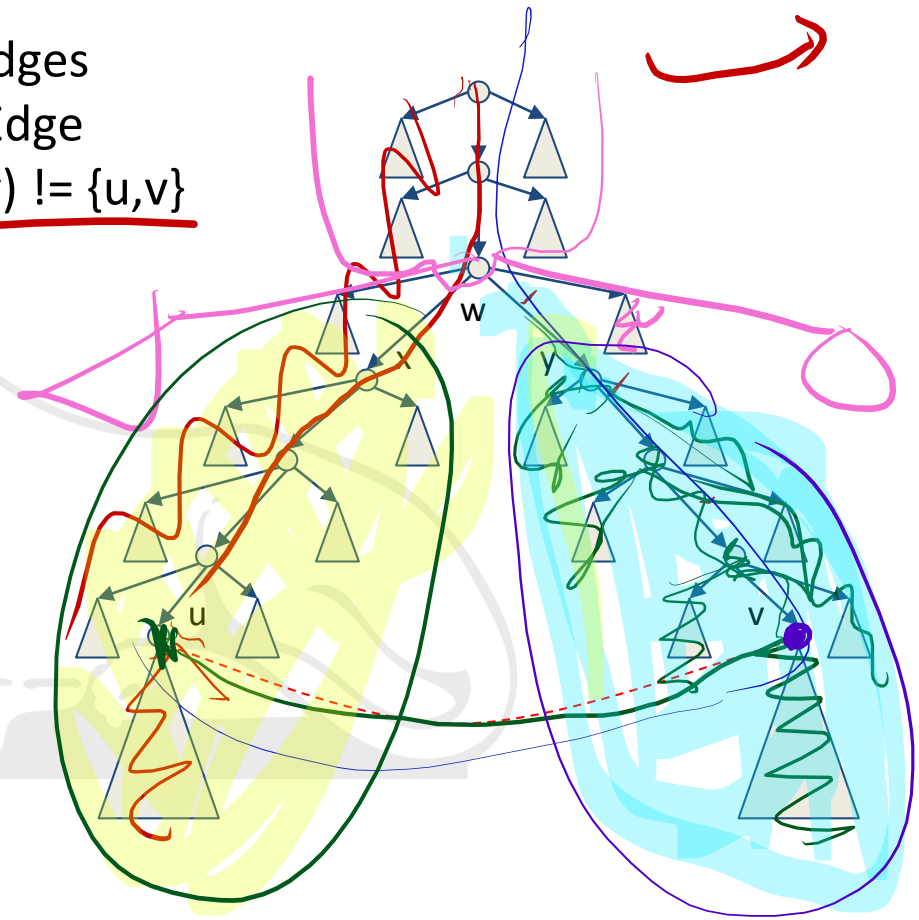
Given an undirected graph G, maintain its DFS under insertion of edges.

Non-Tree edges? Back Edges

Insertion of Edge (u,v) Cross Edge

Detection LCA(u,v) != {u,v}

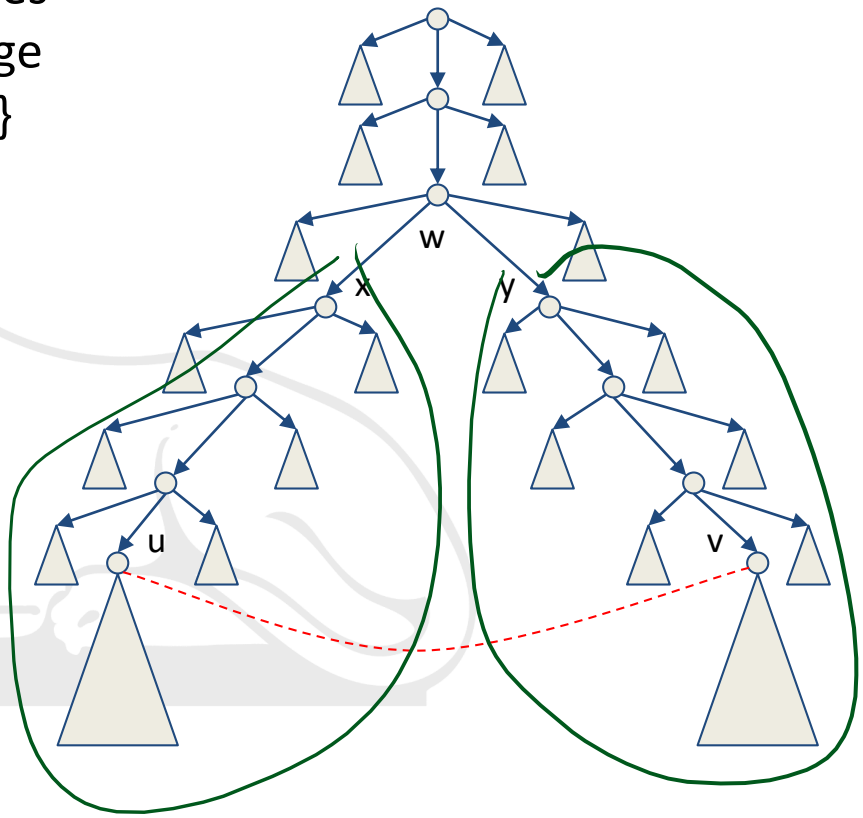
Affected Vertices?



Incremental Depth First Search Trees (Undirected)

Given an undirected graph G, maintain its DFS under insertion of edges.

Non-Tree edges? Back Edges
Insertion of Edge (u,v) Cross Edge
Detection LCA(u,v) != {u,v}



Affected Vertices:
Either Rebuild T(x) or T(y)
or
Child subtree of LCA(u,v)
containing u or v

Incremental Depth First Search Trees (Undirected)

Given an undirected graph G, maintain its DFS under insertion of edges.

Non-Tree edges? Back Edges

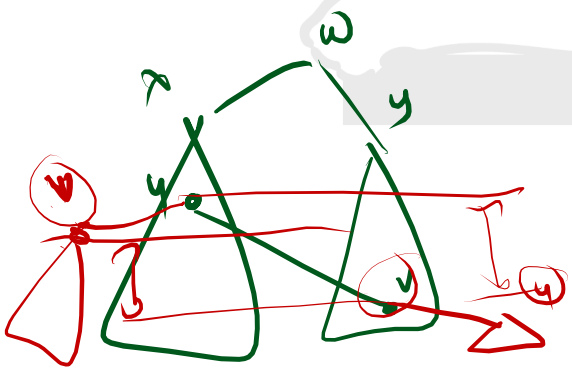
Insertion of Edge (u,v) Cross Edge

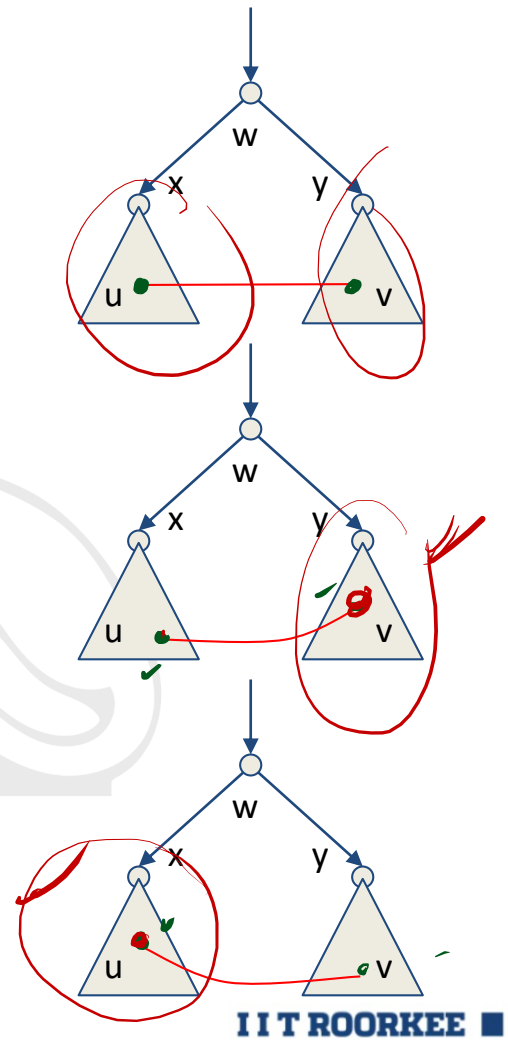
Detection LCA(u,v) != {u,v}

Affected Vertices: T(x) or T(y)

Approach: **Monotonic Fall**

Which to Rebuild T(x) or T(y)?

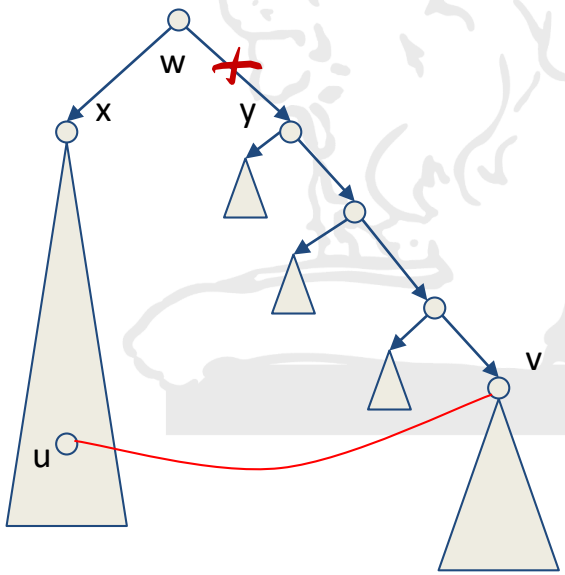




Incremental Depth First Search Trees (Undirected)

Given an undirected graph G, maintain its DFS under insertion of edges.

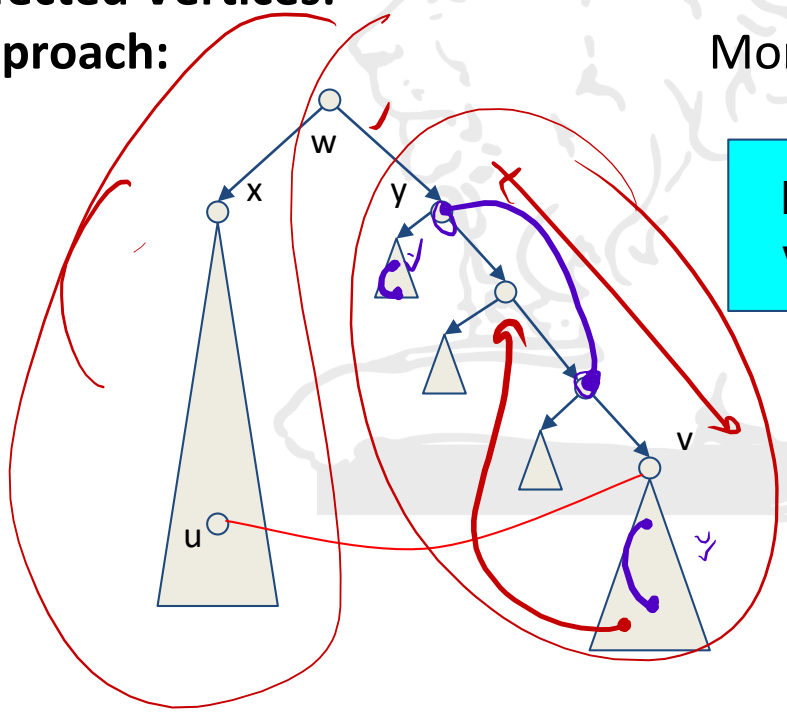
Non-Tree edges?	Back Edges
Insertion of Edge (u,v)	Cross Edge
Detection	LCA(u,v) != {u,v}
Affected Vertices:	T(x) or T(y)
Approach:	Monotonic Fall



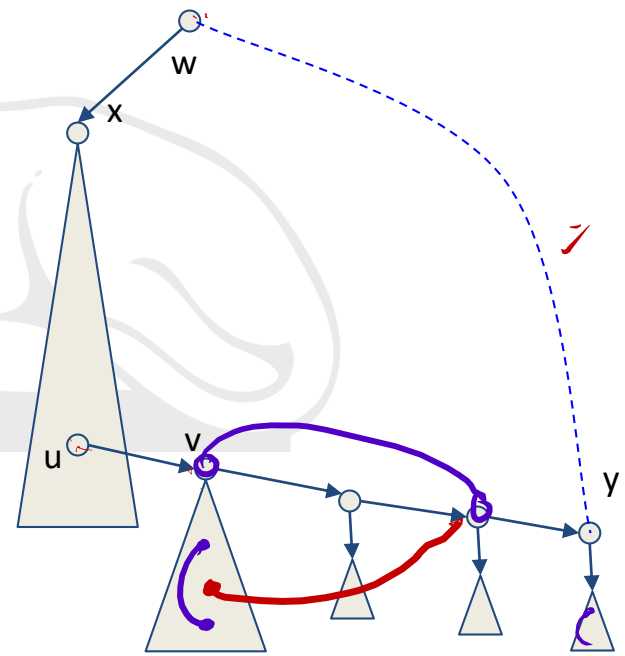
Incremental Depth First Search Trees (Undirected)

Given an undirected graph G, maintain its DFS under insertion of edges.

Non-Tree edges?	Back Edges
Insertion of Edge (u,v)	Cross Edge
Detection	LCA(u,v) != {u,v}
Affected Vertices:	T(x) or T(y)
Approach:	Monotonic Fall



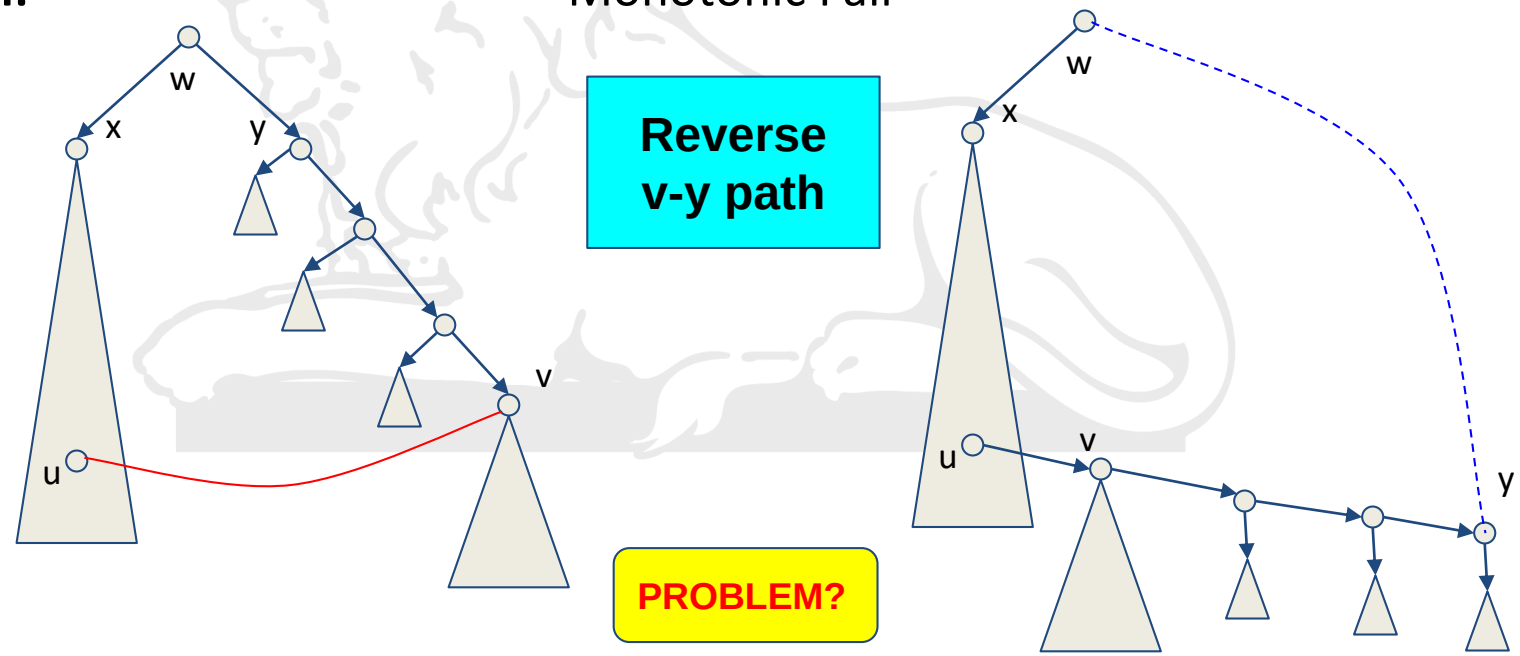
Reverse v-y path



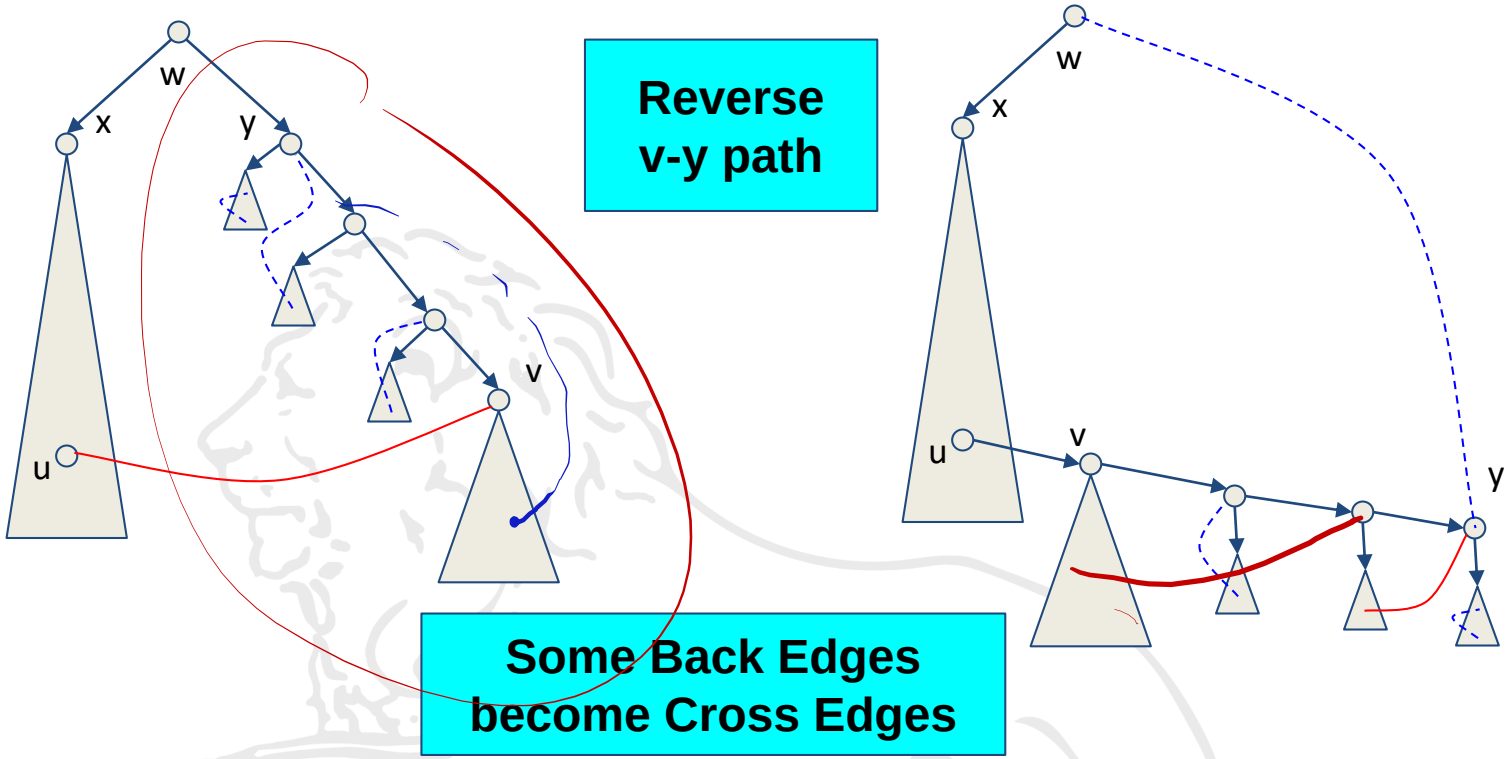
Incremental Depth First Search Trees (Undirected)

Given an undirected graph G, maintain its DFS under insertion of edges.

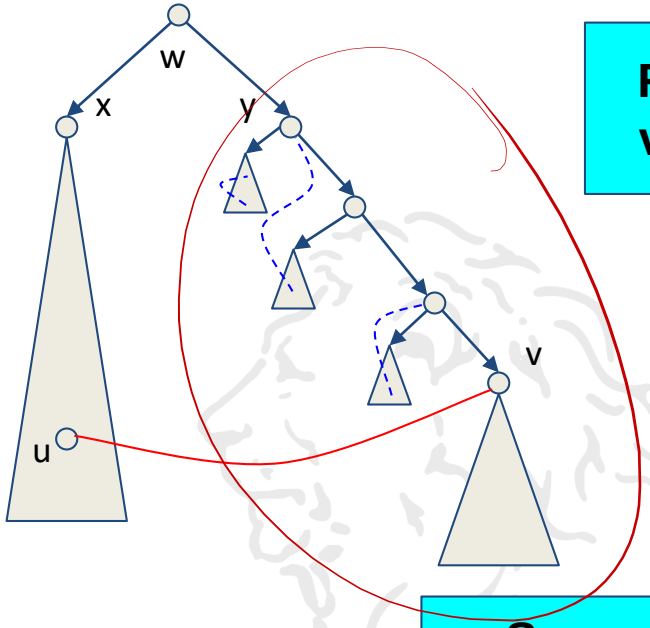
Non-Tree edges?	Back Edges
Insertion of Edge (u,v)	Cross Edge
Detection	LCA(u,v) != {u,v}
Affected Vertices:	T(x) or T(y)
Approach:	Monotonic Fall



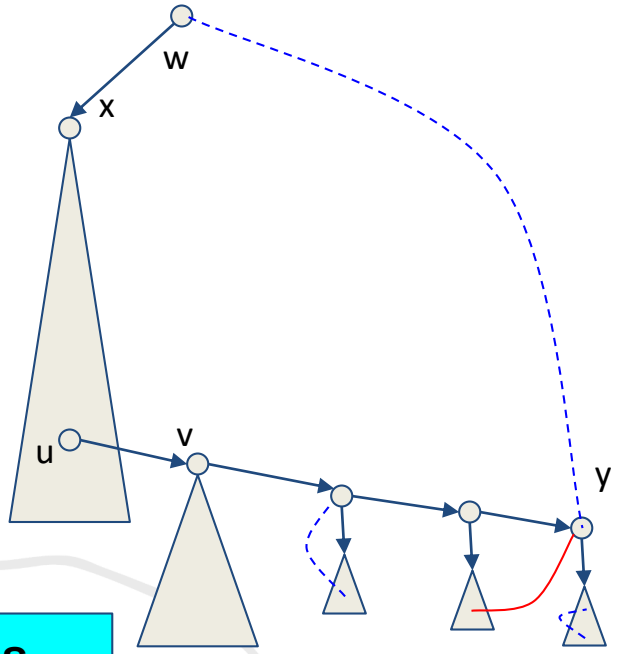
Incremental Depth First Search Trees (Undirected)



Incremental Depth First Search Trees (Undirected)



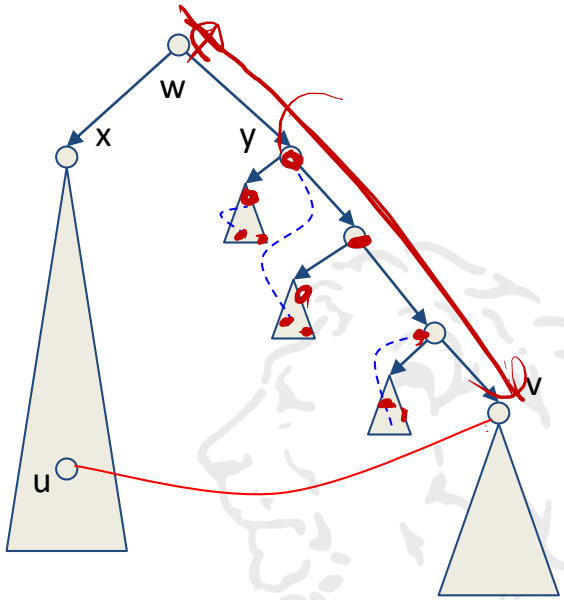
Reverse v-y path



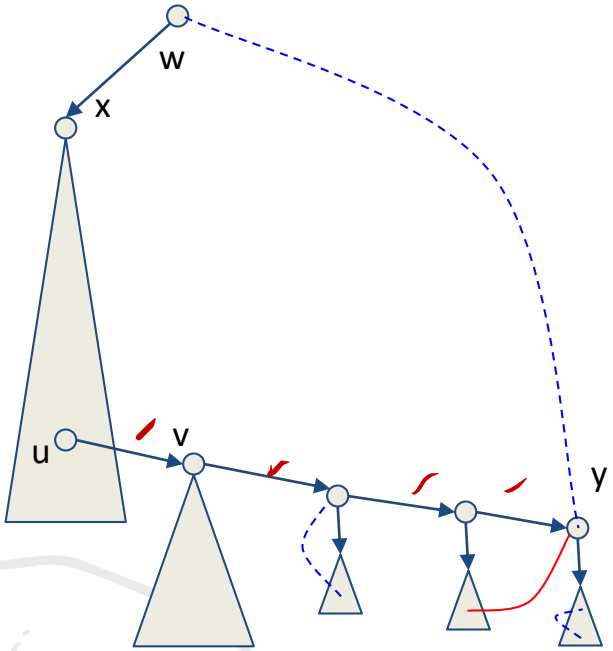
Some Back Edges become Cross Edges

Approach:
Collect the Cross Edges
Insert them one by one

Incremental Depth First Search Trees (Undirected)



Reverse v-y path

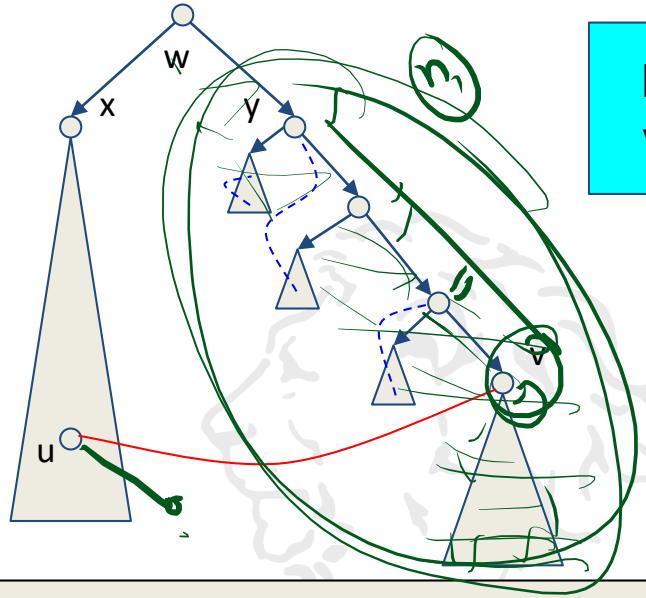


Algorithm:

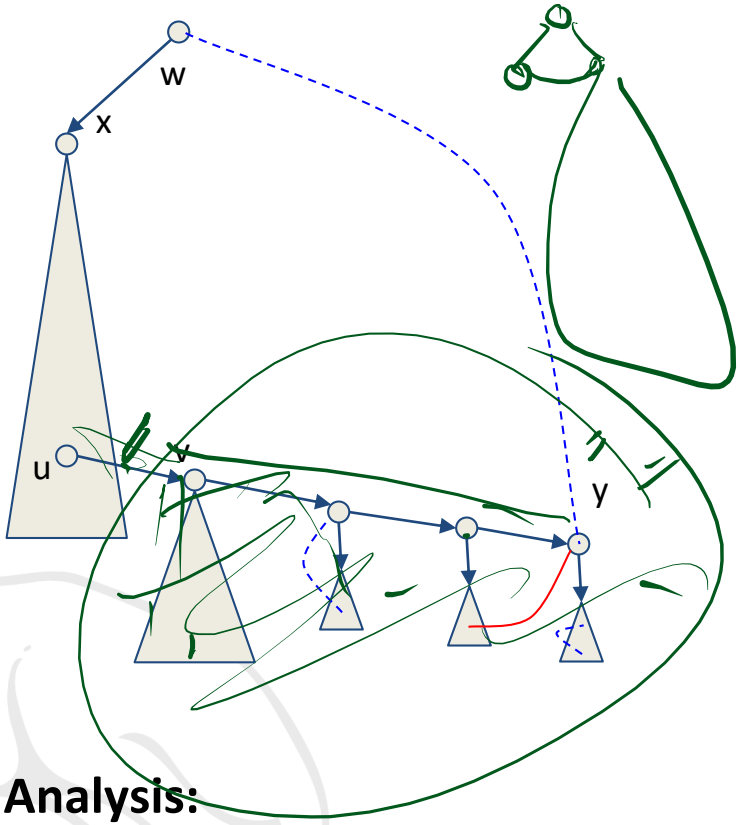
```
InsE = (u,v)
While(InsE not Empty)
    (u,v) = Pop from InsE
    Add (u,v) to G          // lev(u)<=lev(v)
    If( LCA(u,v)!={u,v})
        Reverse (v,LCA(u,v)) except LCA(u,v)
        Add (u,v) to DFS Tree
        Add cross edges of T(v) in InsE
        Update LCA structure
```

Analysis?

Incremental Depth First Search Trees (Undirected)



Reverse v-y path



Algorithm:

```
InsE = (u,v)
While(InsE not Empty)
    (u,v) = Pop from InsE
    Add (u,v) to G          // lev(u)<=lev(v)
    If( LCA(u,v)!={u,v})
        Reverse (v,LCA(u,v)) except LCA(u,v)
        Add (u,v) to DFS Tree
        Add cross edges of T(v) in InsE
        Update LCA structure
```

O(1)

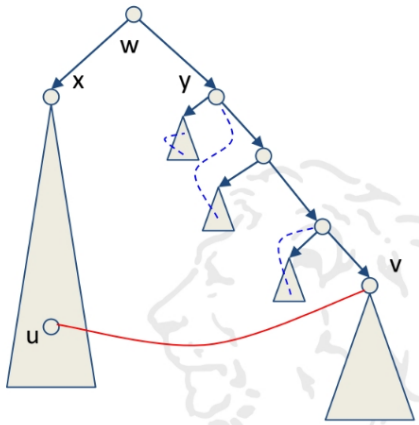
Analysis:

deg(x) whenever x in T(v)

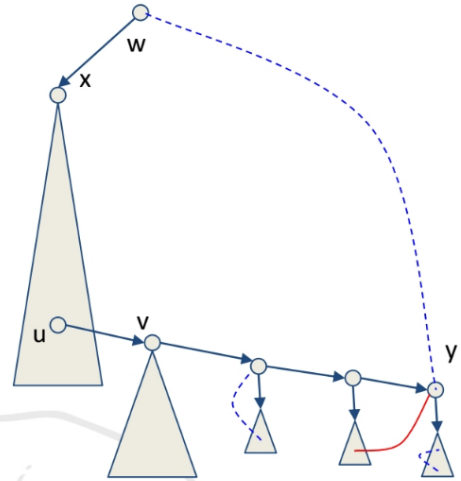
Or deg(x) whenever x falls

WC = O(m+n)

Incremental Depth First Search Trees (Undirected)



Reverse v-y path



Algorithm:

```
InsE = (u,v)
While(InsE not Empty)
    (u,v) = Pop from InsE
    Add (u,v) to G          // lev(u)<=lev(v)
    If( LCA(u,v)!={u,v})
        Reverse (v,LCA(u,v)) except LCA(u,v)
        Add (u,v) to DFS Tree
        Add cross edges of T(v) in InsE
        Update LCA structure
```

Analysis:

deg(x) whenever x in T(v)
Or deg(x) whenever x falls

Total= O(n) ∑ deg(v) = O(mn)
Amortized O(n)

Incremental Depth First Search Trees (Undirected)

Suppose no-cross edges needed to be collected

↳ Cost is $|T(y)|$

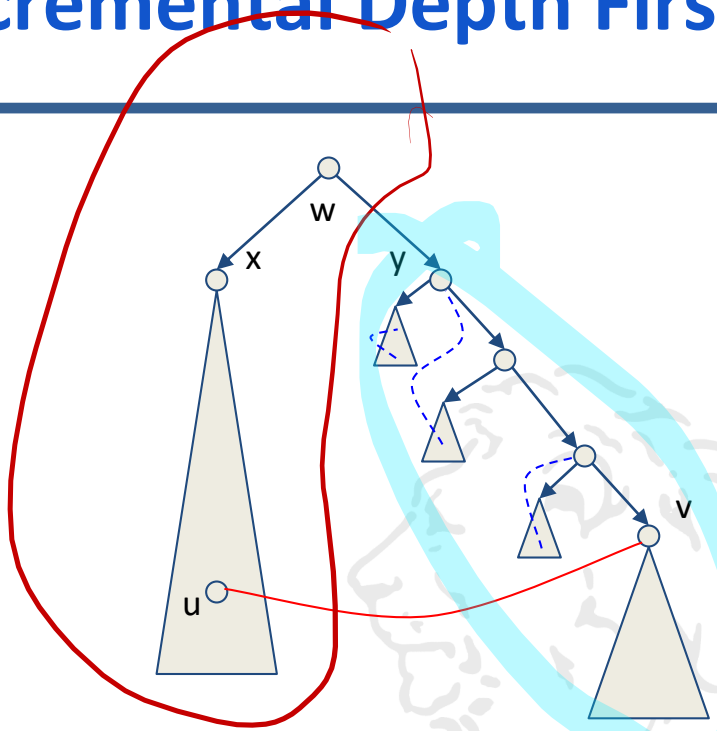
WC $O(n)$

Am - $O(n^2/m)$ → Dense $O(1)$

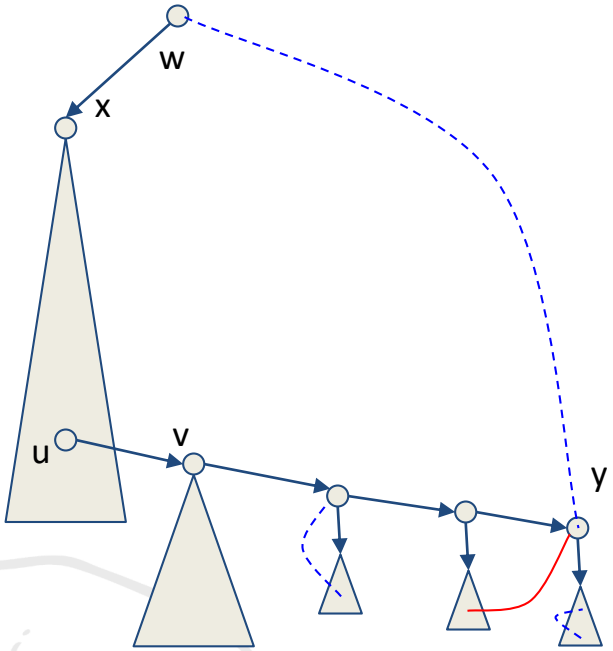
Total $O(n) \times \sum_{v \in V} 1$

$= O(n^2)$

Incremental Depth First Search Trees (Undirected)



Reverse v-y path



Algorithm:

```
InsE = (u,v)
While(InsE not Empty)
        (u,v) = Pop from InsE
        Add (u,v) to G          // lev(u)<=lev(v)
        If( LCA(u,v)!={u,v})
                Reverse (v,LCA(u,v)) except LCA(u,v)
                Add (u,v) to DFS Tree
                Add cross edges of T(v) in InsE
                Update LCA structure
```

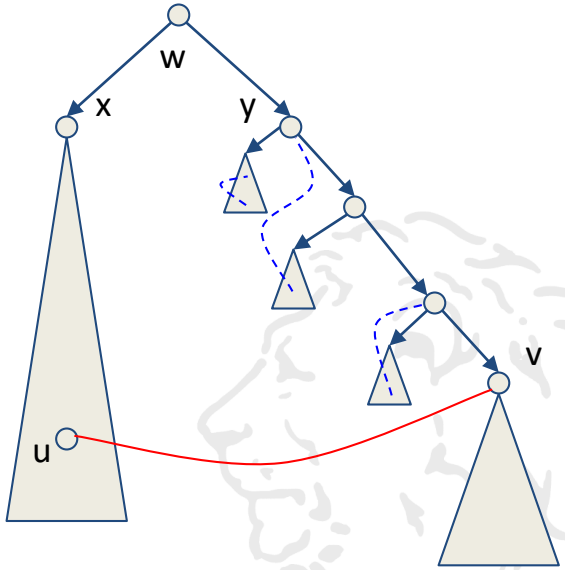
Correctness?

Every non-tree edge is a back edge

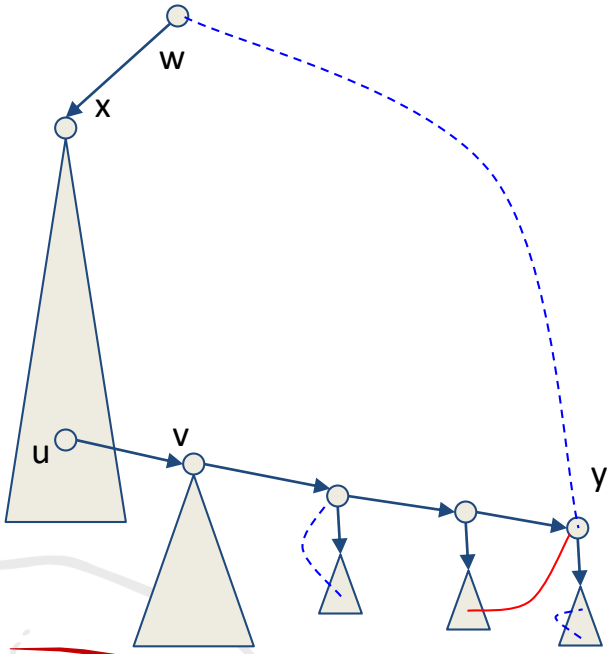
Induction

Inv $G \setminus InsE$ has no cross edge

Incremental Depth First Search Trees (Undirected)

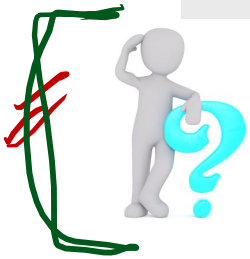


Reverse v-y path



Approach: O(mn)
Collect the Cross Edges
Insert them one by one

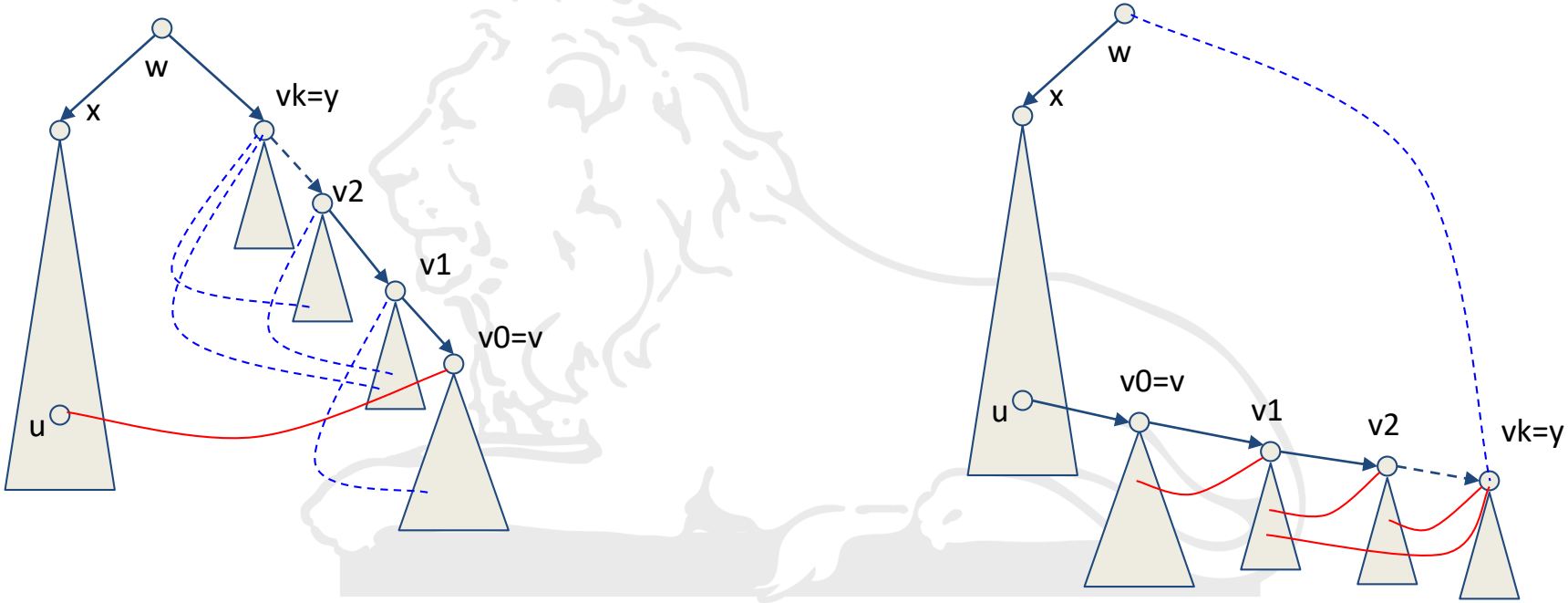
Improve to O(n2)



Think?
How does this cross edge look like?
How many such cross edges?

Back Edges converting to Cross Edge

Think?

How does this cross edge look like?

How many such cross edges?